

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What is the output?

```
try:
    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")
```

- A)A
- B)SyntaxError
- C)A B
- D)B

Q2. What will be printed?

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("A")
except Exception:
    print("B")
```

- A)A B
- B)A
- C)B
- D)Nothing

Q3. What will be the output?

```
x = 10

def outer():
    x = 20

    def inner():
        global x
        x = 30

    inner()
    print(x)
```

```
outer()
print(x)
A)20
10
```

- B)30
- 30
- C)30
- 20
- D)20
- 30

**Q4. if test.py has
print("A")**

```
if __name__ == "__main__":
    print("B")
```

**If test.py is executed directly, what
will be printed?**

- A)A followed by B
- B)Nothing
- C)B
- D)A

**Q5. What does the following
function return?**

```
def fun():
    print("Hello")
```

```
x = fun()
```

```
print(x)
A)Hello
B)None
C>Error
D>Hello None
```

Q6. What is the output?

```
def outer():
    x = 10

    def inner():
        return x + 5

    return inner
```

```
f = outer()
print(f())
```



- A)None
- B)NameError
- C)10
- D)15

Q7. What will happen?

```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

- A)15
- B)SyntaxError
- C)TypeError
- D)25

Q8. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

- A)Module loaded
Module loaded
10
- B)Module loaded
10
- C)ImportError
- D)10
10

Q9. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

- A)TypeError
- B)20
- C)30
- D)SyntaxError

Q10. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

- A)None
- B)20
- C)10
- D)NameError

Q11. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

- A)10
- B)-10
- C)20
- D)30

Q12. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)Nothing
- B)A and B
- C)A
- D)B

Q13. What will be printed?

```
def fun():  
    try:  
        return 10  
    finally:  
        print("Finally")
```

```
print(fun())
```



- A)10 only
- B)Finally only
- C)10 followed by Finally
- D)Finally followed by 10

Q14. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
            numbers)
    )
)
```

```
print(result)
```

- A)[2, 4, 6, 8, 10]
- B)[1, 3, 5]
- C)[2, 4]
- D)[4, 8]

Q15. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)6.0
- B)36
- C)6
- D)NameError

Q16. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)random cannot be imported
- B)Python will always reject it
- C)The local file can shadow the intended standard-library module
- D)Python automatically renames the local file

Q17. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

- A)0
- B)24
- C)10
- D)9

Q18. What will be the output?

```
def fun(x):
```

```
    try:
```

```
        if x < 0:
```

```
            raise ValueError("negative")
```

```
        return x
```

```
    except ValueError:
```

```
        return 100
```

```
    finally:
```

```
        print("cleanup")
```

```
print(fun(-5))
```

- A)cleanup
- 5
- B)100
- cleanup
- C)negative
- cleanup
- D)cleanup
- 100

Q19. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal x
```

```
        x = x + 5
```

```
        return x
```

```
    inner()
```

```
    return x
```

```
print(outer())
```

- A)5
- B)15
- C)UnboundLocalError



D)10

Q20. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

A)20

B)RuntimeError

C)None

D)10

Q21. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

A)Nothing

B)Wrong Value

C)ValueError

D)e

Q22. What is the output?

```
def fun(x):  
    x += 10  
    return x
```

x = 5

y = fun(x)

print(x, y)

A)15 15

B)15 5

C)5 15

D)5 5

Q23. What will be printed?

```
def fun(a, b):  
    return a + b
```

print(fun(10, b=20))

A)SyntaxError

B)TypeError

C)30

D)20

Q24. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)InvalidAgeError

15

B)Nothing, because custom exceptions cannot be caught

C)InvalidAgeError

Age is too low

D)Exception

Age is too low

Q25. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

print(fun())

A)A

B

C

D

10



B)A

C

D

20

C)A

B

10

D

D)A

B

D

10

Q26. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

print(fun(3))

A)TypeError

B)3

C)6

D)RecursionError

Q27. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

variable = 10

fun()

A)10

B)NameError

C)50

D)UnboundLocalError

Q28. What is the output?

```
def fun(x):  
    return x * 2
```

a = fun

print(a(5))

A)10

B)5

C)func(5)

D>Error

Q29. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.modules

B)sys.imports

C)sys.path

D).__module__

Q30. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

A)TypeError

B)MyError

C)Exception

D)Something went wrong

Q31. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

print(fun(3))

A)3

B)6

C)5

D)None

Q32. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It prevents all imports

B)It creates the main function

C)It prevents syntax errors

D)It identifies code intended to run when the file is executed directly

Q33. What will be printed?



x = 10

```
def outer():  
    x = 20
```

```
    def inner():  
        print(x)
```

```
    x = 30  
    inner()
```

outer()

- A)NameError
- B)10
- C)30
- D)20

Q34. What happens here?

```
def fun(a):  
    return a
```

print(fun())

- A)None
- B)NameError
- C)TypeError
- D)0

Q35. What will be the output?

```
import math
```

print(math.sqrt(25))

- A)25
- B)sqrt(25)
- C)5
- D)5.0

Q36. What will be the output?

x = 10

```
def fun():  
    print(x)  
    x = 20
```

fun()

- A)10
- B)UnboundLocalError
- C)20
- D)NameError

Q37. What will happen?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal y  
        y = 20
```

```
    inner()
```

outer()

- A)NameError
- B)10
- C)20
- D)SyntaxError

Q38. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

- A)ZeroDivisionError is not an exception
- B)ZeroDivisionError occurs before Exception
- C)Exception is a superclass of ZeroDivisionError
- D)except blocks execute randomly

Q39. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)
```

- A>Error
- B)[10]
- [10, 10]
- C)[10, 10]
- [10]



D)[10]

[10]

Q40. What will be printed?

```
try:
    x = int("10")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

A)B

B)A

C)B C

D)A C

Q41. What will be printed?

```
def fun():
    print("A")
    return
    print("B")
```

fun()

A)Nothing

B)A B

C)A

D)B

Q42. What happens when this code runs?

```
def first():
    second()
```

```
def second():
    third()
```

```
def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
```

A)RuntimeError cannot propagate through functions

B)TypeError

C)Handled

D)NameError

Q43. What will be printed?

```
def make_counter():
    count = 0
```

```
    def counter():
        nonlocal count
        count += 1
        return count
```

```
    return counter
```

```
a = make_counter()
```

```
b = make_counter()
```

```
print(a())
```

```
print(a())
```

```
print(b())
```

```
print(a())
```

A)1

2

1

3

B)0

1

0

2

C)1

2

3

4

D)1

1

1

1

Q44. What happens when this function is called?

```
def fun(a=10, b):
    return a + b
```

A)TypeError

B)It executes correctly

C)NameError

D)SyntaxError

Q45. What is the output?

```
f = lambda x: x + 5
```



print(f(10))

- A)10
- B)5
- C)15
- D)Error

Q46. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

- A)Error
- B)Only Hello
- C)x
display
- D)10
Hello

Q47. What is the output?

```
try:  
    raise ValueError  
except ValueError:  
    print("A")  
    raise
```

- A)No Output
- B)A, and execution ends normally
- C)A, followed by propagation of the same exception
- D)B

Q48. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

- A)First imports only sqrt
- B)There is no difference

C)First provides the module namespace;
second imports sqrt directly

D)Both import only sqrt

Q49. What is the output?

```
try:  
    x = int("abc")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

- A)A
- B)B C
- C)A C
- D)B

Q50. What will normally happen?

```
import math  
import math
```

- A)ImportError
- B)The module is normally loaded once and reused
- C)The module is normally loaded and executed twice
- D)SyntaxError

